

COS 214 Practical 4

TaskForge: Hierarchical Work Processing

Date Released: 1 September 2026
Date Due: 8 September 2026 at 11:00
Team Size: 3 students
Language: C++11
Core topics: Iterator, Composite, State, Decorator, UML Activity Diagrams, GDB, Valgrind, Docker, Git/GitHub

1 Introduction

Many real systems organise work hierarchically. A large unit may contain smaller groups as well as individual items, and those items may move through different stages during execution. The same hierarchy may need to be inspected in different ways, while some objects may acquire additional responsibilities without changing their underlying class.

In this practical your team will design and implement **TaskForge**, a reusable work-processing system for a domain of your choice. Possible domains include software delivery, construction, logistics, manufacturing, maintenance, film production, research management, emergency response, or another realistic domain approved by your tutor.

You are not given a class structure. You are given the problem and the patterns that must appear meaningfully in your solution. Deciding where each pattern belongs, how the collaborations meet, what is owned, and how the system should behave is part of the practical.

Your solution must make meaningful use of the following Gang of Four patterns:

- Iterator;
- Composite;
- State; and
- Decorator.

The design must be coherent as one system rather than four isolated pattern demonstrations.

2 Learning Outcomes

After completing this practical you should be able to:

- recognise where Iterator, Composite, State and Decorator are appropriate in a larger design;
- separate traversal from the internal representation of an aggregate;
- design and traverse a genuine recursive part-whole structure;
- model runtime behaviour that changes with object state;
- add responsibilities to objects dynamically without creating a large subclass hierarchy;
- reason about ownership, lifetime and changes to a structure during traversal;
- model substantial workflows using UML Activity Diagrams;
- use GDB and Valgrind as part of a debugging workflow;
- build and run the complete project inside Docker; and
- collaborate professionally using Git and GitHub.

3 Non-negotiable Rules

1. Complete this practical in a **team of three**. All three members must understand the complete submission.
2. Compile using `-std=c++11`. The project must be built using a **Makefile**; the marker will build it by running `make`.
3. The final executable must be named `taskforge`.
4. Every polymorphic base class must have a virtual destructor.
5. Every owning relationship must have a clear destruction policy. Your implementation will be checked for memory errors and leaks.
6. Your runtime hierarchy must contain at least **three levels of nesting below the root/client boundary** and must contain both individual objects and nested groups.
7. Client code may not obtain the complete internal container of a group merely in order to traverse it.
8. The required patterns may not be replaced by type checks, giant `switch` statements, large centralised `if/else` chains, or manual client code that knows every concrete class.
9. Your UML and implementation must agree.
10. The complete project must build and run in the supplied Docker environment, including GDB and Valgrind.

GITHUB IS COMPULSORY. A repository created or populated only at the end does not count as meaningful GitHub use. The repository history must show development over time and meaningful contributions from all three team members.

4 Scenario and System Requirements

Your team must invent a system that manages a nested body of work. The hierarchy should be rich enough that groups may contain both smaller groups and individual work items, and that treating a single item and an entire group uniformly is useful somewhere in the application.

The system must support multiple ways of moving through this hierarchy. At least one traversal must visit the complete structure, while another must differ meaningfully in traversal order, selection rule, or purpose. Two traversals of the same structure must be able to exist independently.

Individual work items must have a meaningful lifecycle. Their behaviour must change as they move through that lifecycle, and invalid actions or transitions must be handled sensibly. The lifecycle must be appropriate to your chosen domain rather than copied from an example in the specification.

Your system must also support adding optional responsibilities to suitable objects at runtime. These responsibilities must be stackable in a meaningful way. For example, a domain may need combinations such as auditing, priority handling, cost adjustment, logging, validation, security, notification, insurance, or another responsibility that makes sense for the chosen scenario. The decorated object must still be usable through the same relevant abstraction as the undecorated object.

During execution, your system must undergo at least one meaningful structural or behavioural change. Examples include moving an item between groups, adding/removing an item, changing a lifecycle state, or adding/removing a runtime responsibility. Your team must decide what such changes mean for any traversal already in progress and implement a defensible policy.

Task 1: Design the System

Before implementation, define the domain and complete the design.

Your submitted design must include:

- a concise description of the chosen domain and the problem your system solves;
- a complete UML class diagram drawn from scratch;
- identification of the GoF participants for Iterator, Composite, State and Decorator in your own design;
- relevant inheritance, associations, multiplicities and ownership relationships;
- enough concrete domain classes to make the hierarchy and runtime behaviour non-trivial; and
- a short rationale for the important design and ownership decisions your team made.

Do not design toward a fixed number of classes. The design should be as small as it can reasonably be while still supporting the required behaviours.

Task 2: Implement the Core Model

Implement the object model described by your UML.

Your implementation must demonstrate all of the following somewhere in the running system:

- a genuine recursive part-whole hierarchy with meaningful leaf and grouping behaviour;
- traversal without exposing the hierarchy's internal representation to client code;
- at least two meaningful traversal behaviours over the same hierarchy;
- two independent traversals over the same runtime structure;
- state-dependent behaviour with valid and invalid transitions;
- at least two meaningful decorators that can be applied at runtime, including a case where decorators are stacked; and
- sensible ownership and clean polymorphic destruction.

Using STL containers internally is allowed. Exposing a container to `main.cpp` and treating an STL loop as the required Iterator solution is not.

Task 3: Dynamic Behaviour and Design Decisions

Your system should behave like an application rather than a static pattern demonstration.

Create at least two non-trivial runtime scenarios in which several parts of the design collaborate. Across these scenarios, your program must include:

- traversal of nested objects;
- lifecycle/state-dependent behaviour;
- at least one decorated object participating in normal system behaviour; and
- at least one runtime change to the structure, state, decoration, or a combination of these.

Your team must decide and document how an existing traversal behaves when the structure it refers to changes. Snapshot traversal, live traversal, invalidation, or another policy may be used, but the policy must be deliberate, safe and consistent with the implementation.

Task 4: UML Diagram Portfolio

Submit a diagram portfolio that explains the design and the important workflows in your actual system.

In addition to the class diagram from Task 1, include:

- one UML object diagram showing a meaningful runtime portion of your nested hierarchy;
- one UML state diagram showing the lifecycle of an important stateful object; and

- **three substantial UML Activity Diagrams** chosen to explain the most important workflows in your system.

The Activity Diagrams should not all tell the same story. Across the three diagrams, demonstrate appropriate use of actions, decisions, guards, merges and loops, as well as at least one fork/join, one called/composite activity, and one use of swimlanes.

At least one Activity Diagram must make traversal visible as part of a larger workflow rather than hiding the entire traversal behind a single “process all” action. At least one should contain meaningful conditional behaviour related to the lifecycle or runtime configuration of an object. The third is your team’s choice and should showcase a distinctive workflow from your domain.

The diagrams are part of the design. Important actions and decisions shown in the diagrams must be traceable to the final C++ design and behaviour.

Task 5: Debugging and Memory Investigation

Use the debugging tools introduced in the module as part of development, not only after the program is finished.

Your PDF must contain a short engineering investigation that includes:

- evidence of using GDB on a meaningful part of your program, including breakpoints/stepping and inspection of relevant program state;
- one genuine bug encountered during development, with its symptom, cause, debugging evidence and correction; and
- Valgrind evidence for the final application.

The final implementation must not contain definitely-lost memory originating from your own code.

A typical Valgrind command is:

```
valgrind --leak-check=full --show-leak-kinds=all ./taskforge
```

Task 6: Docker and GitHub Workflow

The repository is both the development environment and evidence of the team’s engineering process.

Your repository must contain at least:

- all source/header files and `main.cpp`;
- a working `Makefile`;
- a working `Dockerfile`;
- `README.md`; and
- a `docs/` directory containing the submitted design material or exported diagrams.

The Docker environment must provide the tools needed to compile, run and investigate the system, including `g++`, `make`, `gdb` and `valgrind`. The `README.md` must contain sufficient commands for a marker to reproduce the build and execute the program, GDB and Valgrind without installing project-specific dependencies directly on the host machine.

Git history must show genuine work by all three members across the development period. Use meaningful commit messages. Branches, pull requests, issues and reviews are encouraged where useful, but the important requirement is a credible collaborative history rather than a particular Git workflow.

Include a short reflection in the PDF explaining how work was divided and integrated, with references to a few commits, pull requests, issues or other repository activities that demonstrate collaboration.

Task 7: Integration and Demonstration

Prepare a coherent demonstration of approximately five minutes. Do not structure the program as a menu that simply runs one pattern example after another. The demonstration should tell a believable story in your chosen domain and naturally expose the important design decisions.

During the demonstration, a tutor may ask any team member to:

- identify where a GoF participant appears in the code;
- explain an ownership or lifetime decision;
- explain how traversal state is maintained and what happens after a runtime modification;
- trace an Activity Diagram action or decision to the implementation;
- explain a state transition or decorated behaviour;
- run or discuss GDB and Valgrind evidence;
- build/run the program in Docker; or
- discuss the team's Git history.

All three team members are responsible for understanding the complete submission.

5 Submission

Submit one source archive and one PDF. The GitHub repository must remain accessible to the teaching team until marking is complete.

The PDF should contain, in a sensible order:

- system description and team details;
- completed UML class diagram and GoF participant mapping;
- concise design/ownership rationale;
- object diagram and state diagram;
- the three Activity Diagrams;
- the team's traversal-modification policy and any other important design decisions;
- GDB bug investigation and Valgrind evidence;
- GitHub repository link and workflow reflection; and
- a short team contribution statement.

6 Final Advice

Begin with the problem, not the pattern class names. Decide what objects exist in your domain, what belongs inside what, what changes during execution, which responsibilities may be added dynamically, and which workflows need to traverse the structure.

Then decide how the required patterns help solve those problems.

The strongest submissions will be the ones in which the UML, ownership policy, runtime behaviour, C++ implementation and development history tell the same coherent story.